

TECHNICAL DOCUMENTATION

NutriTech

AI-Powered Nutrition Platform — FastAPI + React

YOLOv8 Food Detection

Gemini RAG Chatbot

K-Means + KNN Meal Planner

Macro Tracker

Free / Premium Tiers

Graduation Project · Full-stack implementation reference

This document explains **what every part does** and **how it works internally**, so the team can work on the codebase without needing a walkthrough.

Contents

1. System Overview
2. Architecture & Request Lifecycle
3. Technology Stack
4. Repository Structure
5. Data Layer — Database, Datasets & Seeding
6. Authentication, Security & Premium Gating
7. Feature — Macros Calculator
8. Feature — Meal Planner (deep dive)
9. Feature — Food Image Recognition
10. Feature — AI Chatbot (RAG)
11. Feature — Food Search
12. Feature — Daily Tracker & Dashboard
13. Premium Subscription System
14. Frontend Architecture
15. API Reference
16. Setup & Running
17. Known Limitations & Honest Notes

1. System Overview

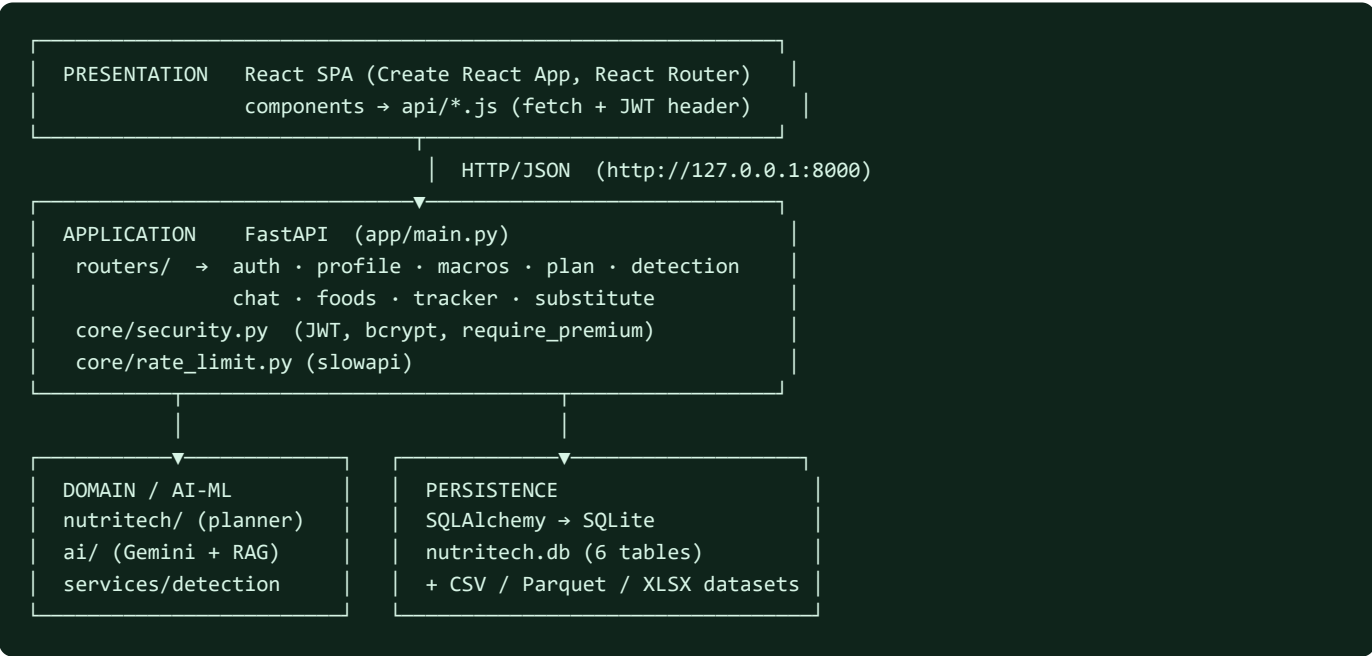
NutriTech is a web application that helps users manage their nutrition. It combines classic nutrition math with three AI/ML subsystems. A user creates a profile (age, weight, height, activity, goal, diet, health flags), and the app then powers seven features:

Feature	What it does	Core technique	Tier
Macros Calculator	Computes daily calorie + macro targets from the profile	BMR/TDEE formulas	Free
Food Search	Look up any food's macros	USDA API + curated dataset	Free
Meal Planner (daily)	Builds a personalized day of meals on target	K-Means + KNN + portion solver	Free
Profile & Tracker	Logs eaten food, shows adherence dashboards	SQL aggregation + Recharts	Free
AI Chatbot	Conversational nutrition assistant	Gemini LLM + RAG	Premium
Image Recognition	Detects food in a photo, estimates macros	YOLOv8 object detection	Premium
Weekly Meal Plan	Generates a full 7-day plan	Meal planner × 7 seeds	Premium

The system is deliberately split so the **nutrition engine** (the meal planner) is a self-contained package (`app/nutritech/`) with no web dependencies — it can be unit-tested or reused independently of FastAPI.

2. Architecture & Request Lifecycle

NutriTech is a classic three-tier application:



2.1 Typical request lifecycle (protected endpoint)

1. React component calls an `api/*.js` helper, which attaches `Authorization: Bearer <JWT>` (from `localStorage`).
2. FastAPI routes the request; the `get_current_user` dependency decodes the JWT, loads the `User` from the DB, and injects it.

- 3. For premium endpoints, `require_premium` additionally checks `user.is_premium` and raises **403** if false.
- 4. `slowapi` enforces a per-IP rate limit (e.g. 15/min for chat).
- 5. The router runs business logic (often delegating to `nutritech/`, `ai/`, or `services/`) and returns JSON.

2.2 Application startup (`app/main.py`)

- `Base.metadata.create_all()` — creates any missing tables.
- `seed_food_if_empty()` — seeds the `food_items` table from the Excel source the first time only (so a fresh clone works with no manual step).
- The meal-planner `FoodStore` is warmed once at import (loads the CSV, fits K-Means + KNN) so the first plan request is fast.
- CORS is open (`*`) for local development.

3. Technology Stack

Layer	Technology	Role
Frontend	React 18 (CRA), React Router, Recharts	SPA, routing, charts
API	FastAPI, Uvicorn, Starlette	HTTP server & routing
Auth	python-jose (JWT), passlib + bcrypt	Tokens & password hashing
Rate limiting	slowapi	Per-IP throttling
ORM / DB	SQLAlchemy 2.0, SQLite	Persistence
ML — detection	Ultralytics YOLOv8, OpenCV	Food object detection
ML — planner	scikit-learn (MiniBatchKMeans, NearestNeighbors), pandas, numpy	Clustering, substitution, scoring
LLM	Google Gemini (<code>google-genai</code>)	Chatbot generation
Data I/O	pandas, pyarrow (parquet), openpyxl (xlsx), httpx	Datasets & external APIs

4. Repository Structure

```
Backend/
  app/
    main.py          FastAPI app, router wiring, startup seeding
    database.py      SQLite engine + session factory
    core/
      security.py    JWT, bcrypt, get_current_user, require_premium
      rate_limit.py  slowapi limiter instance
    routers/         one module per resource (see API reference)
    models/          SQLAlchemy ORM models (users, profiles, logs, plans, foods)
    schemas/         Pydantic request/response models + validation
    services/
      detection_service.py  YOLOv8 wrapper + macro lookup
      macros_service.py
  ai/
    gemini_helper.py  Gemini client, key rotation, prompt assembly
    rag_service.py    parquet food index search + ask_chatbot()
  nutritech/
    core/config.py    ← self-contained meal-planner engine (no web deps)
    models/user.py    constants: roles, templates, splits, caps, expansions
    models/goal.py    BMI / BMR / TDEE / calorie + macro targets
    models/goal.py    rule-based goal engine + goal mapping
    services/data_loader.py  FoodStore: load → cluster → KNN (singleton)
    services/scoring.py  eligibility filter + macro-fit scoring
    services/planner.py  day structure + portion solver
    services/knn_substitute.py  food swap suggestions
    utils/pipeline.py  orchestrates the full plan
  AI_Models/best.pt  YOLOv8 trained weights
  dataused/Food_Macros.xlsx  seed source for food_items
data/
  foods_curated.csv  257-food planner dataset (per-100g, tagged)
  index.parquet      chatbot RAG food index
  sample.parquet     chatbot RAG sample table
seed_db.py           seeds food_items (auto-runs on startup if empty)
nutritech.db         SQLite database (committed, with seed data)
requirements.txt
frontend/
  src/components/    UI (one file per page/widget) + CSS modules
  src/api/           fetch clients (one per backend resource)
  src/consts/        shared vocab (restrictions, allergens)
  src/api/config.js  single source for the backend base URL (deploy-friendly)
```

5. Data Layer — Database, Datasets & Seeding

5.1 SQLite database (`nutritech.db`) — 6 tables

Table	Purpose	Key columns
<code>users</code>	Accounts	name, email (unique), hashed_password, is_active, is_premium (<code>nationality</code> column retained but optional — no longer collected at registration)
<code>user_profiles</code>	One profile per user (1:1)	age, gender, height, weight, activity_level, general_goal, diet_type, diabetes, hypertension, cuisine_pref, dislikes, allergies
<code>daily_logs</code>	Tracker entries	user_id, date (YYYY-MM-DD), name, grams, calories/protein/carbs/fat, source, created_at
<code>saved_plans</code>	Last generated plan per user	user_id, plan_json (daily), weekly_json
<code>macros_history</code>	Snapshots of macro targets	user_id, calories/protein/carbs/fats, created_at
<code>food_items</code>	Detection nutrition lookup	name, serving_size_g, {calories,protein,carbs,fat,fiber}_{min,max}

The first four tables are linked by `user_id`. The DB is committed with seed data; missing tables are auto-created on startup.

5.2 Why two separate food datasets (by design — do not merge)

Source	Used by	Shape	Why separate
<code>food_items</code> table (from <code>Food_Macros.xlsx</code>)	Image recognition	~32 foods, min/max macro <i>ranges</i>	Detection returns a class label; we report a nutrition <i>range</i> because portion size from a photo is uncertain.
<code>data/foods_curated.csv</code>	Meal planner	257 foods, exact per-100g + role/cuisine/diet tags	The planner needs single-ingredient <i>components</i> with roles to compose meals and run clustering/KNN.
<code>data/index.parquet</code> + <code>sample.parquet</code>	Chatbot RAG	Large food index with description + macros	Fast substring search to ground the LLM in real numbers.

5.3 Auto-seeding (`seed_db.py`)

`seed_food_if_empty()` runs on startup: it creates the table and, only if `food_items` is empty, reads `Food_Macros.xlsx` (skipping the descriptive header row) and inserts the foods. Because the committed DB already contains the rows, this is a no-op for the team — but a fresh database is populated automatically.

6. Authentication, Security & Premium Gating

6.1 Passwords

Registration (`schemas/auth.py`) requires **8–72 chars** with at least one lowercase, uppercase, digit, and symbol. Passwords are hashed with **bcrypt** (`core/security.py` via passlib); plaintext is never stored.

6.2 JWT tokens

- On login, `create_access_token(subject=email)` signs a token with `JWT_SECRET_KEY` (HS256), expiring in 24h.
- The secret is read from `.env` — never hardcoded. If missing, an ephemeral key is generated (everyone is logged out on restart) as a nudge to set it.
- `get_current_user` decodes the token, extracts the email (`sub`), and loads the user. Invalid/expired → **401**.

6.3 Premium gating

```
def require_premium(current_user = Depends(get_current_user)) -> User:
    if not current_user.is_premium:
        raise HTTPException(403, "Premium subscription required")
    return current_user
```

This single dependency gates the three premium endpoints (`/chat/ask` , `/detection/detect` , `/plan/weekly*`). Enforcement is on the **server** — hiding the buttons in the UI is not enough, so the API itself rejects free users.

6.4 Rate limiting

slowapi applies per-IP limits: chat **15/min**, detection **20/min**, daily plan **20/min**, weekly plan **8/min**, food search **30/min**. Exceeding returns **429**.

7. Feature — Macros Calculator

Computes daily energy + macronutrient targets from the profile. The pipeline:

1. **BMR** (Mifflin–St Jeor, in the planner engine):

Men: $10 \cdot W + 6.25 \cdot H - 5 \cdot A + 5$ · Women: $10 \cdot W + 6.25 \cdot H - 5 \cdot A - 161$

2. **TDEE** = BMR × activity factor (sedentary 1.2, moderate 1.55, active 1.725).

3. **Calorie target** from the weight goal: loss/keto = $TDEE - 500$, gain = $TDEE + 350$, maintenance = $TDEE$. Clamped to a safe floor/ceiling (1200–4500 kcal, gender-aware floor).

4. **Macro split** by diet (overridden by the diabetes flag): diabetes 25/40/35, keto 25/5/70, high-protein 35/35/30, balanced 20/50/30 (P/C/F %). Grams = kcal·ratio ÷ {4,4,9}.

Implementation note for the team: there are *two* calorie calculators. `nutritech/models/user.py` (**Mifflin–St Jeor**) drives the **meal planner**. `routers/profile.py → calculate_daily_needs()` (**Harris–Benedict**) drives the `/profile/needs` endpoint and the tracker's target line. They agree closely but are not identical — keep this in mind if numbers differ slightly between the planner and the dashboard.

8. Feature — Meal Planner (deep dive)

This is the most sophisticated subsystem. The core idea: **foods are single-ingredient "components" tagged with a role; a meal is assembled from roles, and the exact gram amounts are SOLVED so each meal — and therefore the whole day — lands on its calorie target by construction, while a macro-aware fit keeps protein/carb/fat close to the diet's split.** This replaced an earlier 960-line cascade of hand-written rules.

8.1 The dataset: components, roles & tags

Each of the 257 rows in `foods_curated.csv` is one ingredient with per-100g macros and three tag columns that the engine relies on:

Column	Meaning	Values
<code>role</code>	What part of a meal this food plays	main (protein anchor), carb, veg, fruit, fat, side
<code>cuisine</code>	Cuisine family	egyptian, intl, ...
<code>diet_tags</code>	Which goals the food is eligible for	wl (loss), wg (gain), balanced, keto, diab, hp (high-protein)
<code>item_kind</code>	component vs dish	nearly all are <code>component</code> (single ingredients), enabling reliable name-based exclusion for allergies/dislikes

The "components" idea. Because every food is a single ingredient with a role, the planner can treat a meal as a small recipe: pick a **main** (protein), a **carb**, and a **veg/fruit/fat**, then scale each. This is far more flexible than a fixed list of whole dishes, and it makes allergy/restriction handling trivial — excluding "peanut" just filters any component whose name contains it.

8.2 FoodStore — load, cluster, index (built once)

`data_loader.py` builds a cached singleton `FoodStore`:

1. **prepare()** — coerces numeric columns, then derives macro-energy ratios (`protein_ratio = 4·protein ÷ kcal` , similarly carbs/fat) so foods are comparable regardless of calorie density. Drops zero-calorie rows.
2. **K-Means** (`MiniBatchKMeans` , 8 clusters) on standardized macros + ratios. Each food gets a `cluster` id — used later to enforce **variety** (don't fill a day from one cluster).
3. **KNN** (`NearestNeighbors` , Euclidean on standardized macros) — used for **substitutions** (nearest nutritional neighbour).

8.3 The orchestration pipeline (`utils/pipeline.py`)

```
payload → build_user_profile()      # BMI, normalized fields
        → predict_general_goal()    # rule-based on BMI (if goal='auto')
        → map_to_final_goal()       # (goal × diet) → 1 of 8 final goals
        → calc_daily_calorie_target() # calories + macro grams (diabetes overrides)
        → score_candidates()        # eligible foods + 'score' column
        → [pool-starvation guard]   # enough mains? else 400 error
        → build_daily_plan()        # structure → solver → normalize
```

8.4 Goal engine (rule-based)

`models/goal.py` infers the weight goal from BMI when the user selects "auto" ($<18.5 \rightarrow \text{gain}$, $\geq 25 \rightarrow \text{loss}$, else maintain), then maps (**general goal** $\times$ **diet type**) to one of eight final goals (e.g. `weight_loss_high_protein`, `keto`, `maintenance_balanced`). Diabetes/hypertension are **health flags**, not goals — they influence scoring and macros separately.

Honest note: an earlier version advertised a "Decision Tree" but it was trained on 12 synthetic rows. It was replaced by this transparent **rule-based engine** — which is what Seminar 2 actually describes.

8.5 Scoring — which foods are eligible & how good they are (`scoring.py`)

1. **Diet-tag eligibility:** keep foods whose `diet_tags` contain the goal's tag (e.g. `wl`). If fewer than 12 survive, the filter is relaxed so the planner still has options.
2. **Sodium cap:** hypertension $\rightarrow$ keep only foods ≤ 500 mg/100g.
3. **Exclusions:** allergies are a **hard** exclusion (never relaxed); dislikes are **soft**. Both expand a token to many name terms (e.g. "vegan" $\rightarrow$ all meat/poultry/fish/dairy/egg terms).
4. **Macro-fit score:** a vectorized linear formula per goal. Examples:

- weight-loss: `2.0·protein + 1.2·fiber - 0.03·kcal - 0.30·sugar - ...`
- keto: `1.6·fat + 1.0·protein - 0.8·carbs - 0.5·sugar - ...`
- diabetes: `2.0·fiber + 1.2·protein - 1.2·sugar - 0.15·carbs - ...`

Scores are shifted positive; Egyptian foods get a mild $\times 1.10$ boost. (The engine also supports a stronger $\times 1.35$ cuisine-preference boost, but the cuisine selector was removed from the UI, so plans now always use the mild baseline boost.)

8.6 Day structure & calorie splits (`config.py`)

The user picks **2–6 main meals** and **0–4 snacks**. `build_day_structure()` produces an ordered list of slots (the first main meal is always "breakfast"; snacks are woven between meals). `build_meal_splits()` assigns each slot a share of the day's calories:

- Each snack takes 10% (capped at 40% total); the rest is split across main meals by weight.
- Meal weights breakfast/lunch/dinner = 1.0/1.33/1.0 — which exactly reproduces the classic 30/40/30 three-meal split, but now generalizes to any meal count.

8.7 Composing a meal — roles, templates & the breakfast rule (`planner.py`)

Each slot has a **role template** (e.g. breakfast = [main, carb, fruit, fat]; lunch/dinner = [main, carb, veg, fat]) — every main meal now carries a dedicated **fat** component so the planner can hit the diet's fat target directly instead of relying on incidental fat. Templates are rewritten for the goal: on **keto**, carb $\rightarrow$ fat and fruit $\rightarrow$ veg; on **diabetes**, fruit $\rightarrow$ veg. For each role the solver `_pick()` s a food:

- From foods of that role not already used, capped to ≤ 2 per K-Means cluster (variety), preferring high-protein mains for HP goals, then takes the top-K by score and samples one (controlled randomness $\rightarrow$ variety between regenerations).
- **Breakfast main** is special: dense meats/fish are excluded and cheeses/plain-yogurt are allowed, then a protein **family** (egg / legume / cheese / yogurt) is chosen at random and the best food within it is picked — so breakfasts stay light and varied instead of always defaulting to the highest-scoring cheese.

8.8 The portion solver — the heart of the planner

Given the chosen foods, the meal's calorie target, and the diet's macro split, `_macro_portions()` computes grams per food with a **bounded least-squares fit over four targets at once** — calories plus the protein/carbohydrate/fat energy of the meal — so portions track the macro split, not just total calories:

```
# per-gram energy of each food i (kcal):
cal[i] = kcal/g ; pe[i] = 4·protein/g ; ce[i] = 4·carb/g ; fe[i] = 9·fat/g
A = [cal, pe, ce, fe] # 4 × n
b = [target_cal, p%·target_cal, c%·target_cal, f%·target_cal]
g = lstsq(A, b) # grams best matching all four targets
g = clip(g, lo, hi) # realistic per-food bounds
lo[i] = serving · 0.4
hi[i] = min(serving · 2.5, role_cap) · cap_mult
```

- Because the fit uses each food's **actual** per-gram macros, the incidental protein/fat in carb or "lean" foods is accounted for — this is what pulls protein and fat from ~30% off-target (under a calorie-only solve) down to ~13%, while calories stay ~0.4% off.
- A **role-based calorie budget** (derived from the macro split) seeds the solve and is the fallback if the matrix is singular. Single-item meals fall back to the older calorie-only `solve_portions()` iterative allocator.
- **Role caps** (main 300g, carb 250g, veg/fruit 200g, fat 80g...) stop low-calorie produce from being scaled into absurd amounts (e.g. 300g of lemon).
- **Garnishes** (parsley, garlic, lemon...) are dropped before composing so they're never scaled into a main component.
- `cap_mult` = `max(1, 3 ÷ n_main_meals)`: on few-meal days each meal carries more calories, so the caps are loosened proportionally. This fixed a ~34% undershoot on 2-meal days down to ~6%.

8.9 Whole-day normalization

After all meals are built, `_normalize_day()` compares the day's total to the target and, if off by >2%, scales every item by the ratio (still respecting role caps) so the final day lands on target. Self-tests across all meal/snack combinations show worst-case deviation ≤ ~6.5%.

8.10 Substitution — swapping a food (`knn_substitute.py`)

In the daily view the user can swap any food. `get_substitute_options()` finds the food's row, then `knn_find_substitute()` returns the nearest neighbours **of the same role** by standardized macro vector (preferring the same cluster, excluding garnishes and already-used foods). Each option is re-priced to the original item's calories. Choosing one calls `apply_named_substitute()`, which re-solves grams and refreshes the meal totals.

Same exclusions as the planner. The substitute endpoints are authenticated and read the signed-in user's profile, so the suggested swaps honour the same restrictions as a freshly generated plan: **allergies are a hard exclusion** (allergen token expansions) and the user's **dislikes/preferences are filtered out too** (profile values are unioned with any per-plan overrides). A swap will never offer a food the user is allergic to or has excluded.

8.11 Weekly plan

`POST /plan/weekly` runs the same pipeline 7 times with consecutive seeds (one per day), giving natural day-to-day variety, then returns the 7 days plus weekly averages. This is a **Premium** endpoint.

9. Feature — Food Image Recognition Premium

`services/detection_service.py` wraps a trained **YOLOv8** model (`AI_Models/best.pt` , loaded once at import). Flow for `POST /detection/detect` :

1. The router validates the upload first: allowed types (JPEG/PNG/WEBP/BMP), ≤10 MB, non-empty; optional `weights` JSON is validated as a `{food: grams}` map.
2. Bytes → OpenCV image → `model.predict(conf=0.25)` .
3. Detections are grouped by class name with counts.
4. For each class, the matching `food_items` row is looked up (exact, then fuzzy `contains`). Weight = user override if given, else `serving_size × count` .
5. Macros are scaled by `weight ÷ serving_size` and reported as **min-max ranges** (since exact portions can't be known from a photo). Totals are summed across all foods.
6. An annotated image (boxes drawn) is returned as base64 for the UI to display.

10. Feature — AI Chatbot (RAG) Premium

A Retrieval-Augmented-Generation assistant: relevant foods are retrieved from a local index and injected into the prompt so the LLM answers with **real database numbers** instead of hallucinating.

10.1 Retrieval (`ai/rag_service.py`)

`ask_chatbot(message, history)` first calls `_search_foods()` , which substring- matches the query against `index.parquet` (lower-cased description column) and returns the top 8 foods with their macros. Parquet is read once and cached in memory (needs `pyarrow`).

10.2 Generation (`ai/gemini_helper.py`)

- The prompt = system prompt (instructs the model to use DB facts, never invent macros) + **FOOD DATABASE RESULTS** (the retrieved foods) + the last **10** conversation turns + the user message.
- **API-key rotation pool**: keys come from `GEMINI_API_KEY` + `GEMINI_API_KEYS` (comma list). On a quota/429 error the helper rotates to the next key automatically and retries the same prompt.
- **Model fallback**: it tries `gemini-2.5-flash` → 2.0 variants, remembering the one that works. There is **no startup probe** (so booting doesn't burn quota).
- Token budget adapts to the question (meal-plan asks get more tokens).

Conversation history is threaded end-to-end: the frontend stores the running transcript and sends it with each message, giving the bot real memory within the `HISTORY_WINDOW` .

11. Feature — Food Search

`GET /foods/search?q=&limit=` (auth, 30/min). It queries **USDA FoodData Central** (via `httpx` , key `USDA_API_KEY` or `DEMO_KEY`) and merges with a local curated set, returning per-100g and per-serving macros. If USDA is rate-limited or unreachable it degrades gracefully to curated-only results. The UI renders it on a dedicated `/food-search` tab.

12. Feature — Daily Tracker & Dashboard

Implemented under `routers/tracker.py` (prefix `/tracker`) + the `daily_logs` table:

Endpoint	Purpose
<code>POST /tracker/log</code>	Add an eaten item (source: from-plan / search / manual)
<code>GET /tracker/day?date=</code>	Day's entries + totals + target block
<code>DELETE /tracker/log/{id}</code>	Remove an entry
<code>GET /tracker/range?start=&end=</code>	Zero-filled daily series (≤ 92 days) for charts

The Profile page is split into four tabs: **Preferences**, **Current Plan** (read-only view of the saved plan), **Daily Tracker** (progress bars + add/delete entries; can pull items from the saved daily or weekly plan), and **Progress Dashboard** (Recharts line + bar charts with a target reference line, Week/Month toggle). Targets reuse `calculate_daily_needs()`.

13. Premium Subscription System

Two tiers, **Free** and **Premium**. Premium unlocks the Chatbot, Image Recognition, and Weekly Plans. Implementation:

Layer	Piece	Behaviour
DB	<code>users.is_premium</code>	Boolean flag, default false
API	<code>require_premium</code>	403 for free users on the 3 premium endpoints
API	<code>GET /auth/me</code>	Lets the frontend read premium status
API	<code>POST /auth/subscribe</code> / <code>unsubscribe</code>	Mock toggle of the flag (no real payment)
UI	<code>PremiumGate</code> / <code>usePremium</code>	Wraps premium routes; shows a locked-feature card if not premium
UI	<code>/pricing</code> page	Free vs Premium comparison + Subscribe button

The lock is a bordered card (gold padlock, "Get Premium" → `/pricing`) shown around just the feature, not the whole page. In the Meal Planner, only the **Weekly** mode is locked; daily stays free.

14. Frontend Architecture

- **Routing** (`App.js`): React Router; protected routes are wrapped in `RequireAuth` (redirects to `/login` without a token). Premium routes are additionally wrapped in `PremiumGate`.
- **API clients** (`src/api/*.js`): one module per resource. Every client resolves the backend through a single `config.js` (`BASE_URL = process.env.REACT_APP_API_URL || "http://127.0.0.1:8000"`) — no hardcoded URLs remain, so deploying only needs the env var set at build time. `client.js` wraps fetch and injects the JWT header; chat/detection attach it manually but share the same `BASE_URL`.
- **Auth state**: the JWT lives in `localStorage`; an `authchange` window event lets the navbar and gates refresh after login/logout/subscribe.
- **Components**: page-level (Home, Login, Register, Profile, MealPlanner, FoodSearch, ImageRecognition, Chatbot, Pricing) plus shared widgets (MealCard, CheckGrid, Navbar drawer, Footer, ScrollToTop, DocumentTitle). The Profile tabs and tracker live in dedicated `*Tab.js` files.
- **Styling**: a site-wide dark-green "glass" theme (frosted cards over a green background); a global 85% UI scale via `body { zoom: 0.85 }`; premium accents in gold.

15. API Reference

Method & Path	Auth	Purpose
POST /auth/register	—	Create account (name, email, password; password complexity enforced)
POST /auth/login	—	Get JWT
GET /auth/me	JWT	Current user incl. is_premium
POST /auth/subscribe · /unsubscribe	JWT	Mock toggle premium
GET/PUT /profile/me	JWT	Read/update profile
GET /profile/needs	JWT	Daily macro targets
POST /macros/calculate	JWT	Macros without saving
POST /plan/daily	JWT	Generate daily plan
POST /plan/weekly	JWT Prem	Generate 7-day plan
GET/PUT/DELETE /plan/current	JWT	Saved daily plan
GET/PUT /plan/weekly/current	JWT Prem	Saved weekly plan
POST /substitute/ · /options	JWT	KNN food swap (apply / list); enforces the user's allergies + dislikes
POST /detection/detect	JWT Prem	Detect food in image
POST /chat/ask	JWT Prem	Chatbot reply
GET /foods/search	JWT	Food macro lookup
POST /tracker/log · GET /day · DELETE /log/{id} · GET /range	JWT	Daily tracking

16. Setup & Running

```
# Backend
cd Backend
python -m venv venv
venv\Scripts\activate           # Windows
pip install -r requirements.txt
copy .env.example .env          # then fill in keys
uvicorn app.main:app --reload    # http://127.0.0.1:8000 (docs at /docs)

# Frontend
cd frontend
npm install
npm start                       # http://localhost:3000
```

Required env keys (Backend/.env): `GEMINI_API_KEY`, `JWT_SECRET_KEY`. Optional: `GEMINI_API_KEYS` (rotation), `USDA_API_KEY`. The database and model weights are committed, so no extra data setup is needed. After editing backend Python, restart Uvicorn (or run with `--reload`).

Deploying the frontend elsewhere: the API base URL is not hardcoded. Set `REACT_APP_API_URL` to the deployed backend URL at build time (e.g. an `.env.production` entry) before `npm run build`; with no override it falls back to `http://127.0.0.1:8000` for local development. No code changes are needed to point the app at a different backend.

17. Known Limitations & Honest Notes

- **Mock payments:** "Subscribe" just flips `is_premium`; there is no payment gateway.
- **Two calorie formulas:** the planner uses Mifflin–St Jeor; `/profile/needs` uses Harris–Benedict (see §7). Unify if exact agreement is needed.
- **Keto 3-meal plans** can undershoot calories by ~6.5% because the fat role cap (80g) limits how much energy a keto meal can carry — a known trade-off, not a regression.
- **Detection nutrition is a range**, and the model's classes are limited to what `best.pt` was trained on; foods not in `food_items` report counts only.
- **RAG needs `pyarrow`** to read the parquet index (now in `requirements.txt`); without it the chatbot's food lookup fails.
- **SQLite + open CORS** are development conveniences; production would use a real DB and locked CORS.

— End of document —